

Strategic Architecture Report

Enterprise Agentic Systems for Modern Engineering Organizations

By Nathan Maine | December 2025

Enterprise AI is moving from chatbots that answer questions to agents that take actions inside real systems. This report outlines the architectural patterns required to make that shift reliable, auditable, and governable.

Key idea: treat agentic workflows like production systems: deterministic runs, structured evidence artifacts, and governance at the tool boundary.

Table of Contents

Table of Contents

2

Executive Summary

3

Three Core Architecture Patterns

3

1) Model Context Protocol (MCP)

3

2) Cyclic Graph Orchestration

3

3) Adversarial Evaluation

3

Portfolio Overview

4

Agent Suite (5 runnable tools)

4

Architectural Themes

5

Connectivity: Model Context Protocol

5

Orchestration: From Chains to Graphs

5

Evaluation: Judge Agents and Benchmarks

5

Governance: Guardrails and Audit Trails

5

Future Directions

6

Links

6

Executive Summary

Enterprise AI is shifting from assistants that provide answers to agents that execute actions in production environments. That shift is not just about better models. It requires re-architecting how software, data, and operations work together so that agent behavior is measurable, repeatable, and safe.

In practical engineering organizations, agents are beginning to:

- Provision infrastructure and environments
- Run deployments and validate rollout readiness
- Review architecture and assess technical risk
- Generate tests and evaluate coverage gaps
- Validate compliance checks and approval workflows

In multi-step workflows, reliability degrades quickly without orchestration (retries, branching, and state tracking). Voice-driven interactions introduce additional variance compared to text-only systems in real-world conditions. The architecture patterns in this report are designed to address those gaps.

Three Core Architecture Patterns

1) Model Context Protocol (MCP)

MCP standardizes how agents connect to tools, services, and data sources. Instead of building one-off integrations for each agent and each system (the N+1 problem), MCP exposes capabilities as composable servers with consistent interfaces, user-scoped authentication, and least-privilege access.

2) Cyclic Graph Orchestration

Modern agent flows cannot be modeled as brittle linear chains. Real work requires conditional logic, retries, and loops for self-correction. Cyclic orchestration uses a graph model: planner/worker separation, parallel subagents, branching on validation results, and controlled recovery paths when steps fail.

3) Adversarial Evaluation

Powerful agents should be paired with independent critics. A Doer agent executes work; a Judge agent evaluates outputs against criteria such as regulatory rules, technical correctness, safety constraints, and user experience expectations. This separation enables systematic scoring, regression testing, and accountable approvals.

Portfolio Overview

This portfolio is organized as ten Spec-Kit structured agent projects plus a consistent, runnable mini-suite of five tools. Each project reinforces one or more of the core patterns above.

ID	Project	Key Patterns
1	Compliance Validation Agent	Judge, MCP
2	Architectural Design Review Agent	Search, Subagents
3	MCP Conversational Data Access	MCP, Tool Discovery
4	Agentic Workflow Simplifier	Agent Graphs
5	AI Deployment Experience Bot	MCP, Approvals
6	Semantic Test Coverage Agent	Codegen, Mutation
7	Next-Gen Docforce	Search, Diff-aware
8	Performance Test Generator	Codegen, MCP
9	DevEx Bootstrapping Agent	Shell Execution
10	Voice Robustness Testing	Voice, Benchmarks

Agent Suite (5 runnable tools)

To make the architecture concrete, the portfolio includes a runnable mini-suite of tools that share install flow, CLI conventions, and an artifacts-first output contract. Each tool lives in its own repository and can be run independently or chained together as a pipeline.

Tool	Role	Primary Artifact Outputs
aes	Evaluation harness (Doer/Judge)	out/evidence.jsonl; out/runs/<id>.json
ggc	Policy-to-graph compiler	out/graph.json; out/graph.dot
amg	Memory graph + explain paths	out/graph.json; out/graph.dot; explain traces
shaw	Self-healing workflow runner	out/evidence.jsonl; out/runs/<id>.json
tea	Temporal planner + state	out/plan.md; out/state.json

Architectural Themes

Connectivity: Model Context Protocol

MCP acts as the connective tissue between agents and enterprise systems. External systems are encapsulated as MCP servers with user-scoped authentication and least-privilege access. This keeps integrations composable, testable, and governable.

Orchestration: From Chains to Graphs

Agent graphs with loops, retries, planner/worker/judge separation, and parallel subagents replace brittle linear chains. Graph orchestration is the difference between a demo and a production-capable workflow.

Evaluation: Judge Agents and Benchmarks

Every powerful agent is paired with an equally powerful critic. Techniques include LLM-as-a-Judge, mutation testing, and benchmark-style evaluation loops that measure reliability, correctness, and robustness over time.

Governance: Guardrails and Audit Trails

Governance is strongest at the tool boundary. Patterns include separating read-only vs destructive actions, requiring human-in-the-loop approvals for privileged operations, and producing structured logs and evidence artifacts for auditability.

Future Directions

- Tighter policy and governance integration across tool boundaries
- Deeper metrics and observability for agent graphs in production (latency, retries, failure modes, cost)
- More cross-modal agents (voice + text + UI + logs) with robust evaluation harnesses
- End-to-end platformization: turning proof-of-concepts into reusable, composable building blocks

Links

Umbrella repo: <https://github.com/NathanMaine/agent-ai-portfolio>

Agentic Evaluation Sandbox: <https://github.com/NathanMaine/agent-ai-evaluation-sandbox>

Governance Graph Compiler: <https://github.com/NathanMaine/governance-graph-compiler>

Agentic Memory Graph Engine: <https://github.com/NathanMaine/agent-ai-memory-graph-engine>

Self-Healing Agentic Workflows: <https://github.com/NathanMaine/self-healing-agentic-workflows>

Temporal Executive Agent: <https://github.com/NathanMaine/temporal-executive-agent>